



## COMPREHENSIVE TECHNICAL DOCUMENTATION

*Last Updated: 30<sup>th</sup> May 2026*

*Related Expert Advisor: ZeroDelta\_Ticks.mq5*

## TABLE OF CONTENTS

[Document Map](#)

[Glossary](#)

[1. Overview](#)

[2. System Architecture](#)

[3. Design Philosophy](#)

[4. The Robot Matrix \(Operator Control Panel\)](#)

[5. Market Data & Time](#)

[6. Trading Strategy – X1 Momentum Entry](#)

[7. Initial Entry \(IE\) & X-Open Burst](#)

[8. Recovery Trade \(RT\) & IE-RT Twins](#)

[9. End of Day \(EOD\) & Start of Day \(SOD\)](#)

[10. Risk Management](#)

[11. Session Persistence & Restart Recovery](#)

[12. Command Queue & Execution Feedback](#)

[13. Trade History Cache](#)

[14. HTTP API Reference](#)

[15. MetaTrader 5 Expert Advisor](#)

[16. Data Storage & Configuration](#)

[17. Concurrency & Thread Safety](#)

[18. Logging, Monitoring & Alerts](#)

[19. Troubleshooting](#)

[20. Appendix – Code Map \(Maintainer Index\)](#)

## DOCUMENT MAP

### If you want to understand...

### Read sections

What the system does overall

1, 2, 3

How to configure robots

4, 16

Why a trade did or did not fire

6, 7, 18, 19

IE + RT (recovery) twins

8, Appendix C

End-of-day behaviour

9

Daily loss limits

10

Restart / crash recovery

11

HTTP / EA wiring

14, 15

Code locations for debugging

20

## **GLOSSARY**

| <b>Term</b>           | <b>Meaning</b>                                                                                                      |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>IE</b>             | Initial Entry — the main trade the robot opens at X-open.                                                           |
| <b>RT</b>             | Recovery Trade — a pending stop order placed as a “twin” at the IE stop level; intended to recover if IE hits stop. |
| <b>Twin</b>           | The linked IE position + RT pending/position tracked together in the recovery registry.                             |
| <b>X1</b>             | The signal candle — the bar whose size and shape qualify a setup.                                                   |
| <b>X2</b>             | The prior candle — paired with X1 for momentum pattern direction.                                                   |
| <b>X-open</b>         | The open price of the bar after X1 closes — where IE entry is attempted.                                            |
| <b>Magic number</b>   | A numeric tag on MT5 orders so Java can tell which robot owns a trade.                                              |
| <b>Recovery magic</b> | RT magic derived from IE magic: $IE \times 10000 + 3333$ .                                                          |
| <b>Armed</b>          | Pattern passed filters; system waits for X-open to fire entry.                                                      |
| <b>Flatten</b>        | Close open positions and cancel pending orders for a row (EOD).                                                     |
| <b>Latch (DD)</b>     | Daily drawdown limit tripped; blocks new entries until next MT5 day or manual reset.                                |
| <b>RECONCILING</b>    | Startup phase: Java waits for broker trade history before allowing new entries.                                     |
| <b>RRC</b>            | Request–Response–Confirmation — command sent, EA executes, feedback received before next command.                   |

## 1. OVERVIEW

Zero Delta is an automated trading system in two parts.

- A. MetaTrader 5 (MT5) connects to your broker, reads prices, and places or cancels orders. Java (this application) decides *when* those orders should exist, based on candle patterns, your robot settings, and risk rules. They communicate over HTTP on your computer (localhost port 8087): **the EA “asks” Java for the next instruction rather than Java pushing orders into MT5.**
- B. You configure up to several robots (rows) in the Robot Matrix — each row is a symbol, timeframe, and rule set. When conditions match, the system opens an Initial Entry (IE). If recovery is enabled, it also places a Recovery Trade (RT) stop order at the IE stop level. If IE hits take profit, the RT must be cancelled. At end of day (EOD), enabled rows flatten exposure and cancel stragglers.

### System capabilities

- Real-time OHLC ingestion: multiple timeframes per symbol and tick data for spread filters.
- Multi-robot matrix with per-row: magic numbers, symbols, and timeframes.
- X1 momentum pattern detection:

Setup A = X1 Higher High AND Higher Low or Lower High AND Lower Low or,

Setup B = merely Higher Low for Bear X1 or Lower High for Bull X1

- X-open **burst** entry — *repeated* market attempts within a short T.I.F (Time in Force) window at bar open.
- **Deferred RT** - recovery order queued only after IE **position** ticket is confirmed.
- **IE-RT twin registry** - TP/extinguish, orphan detection, delete retry + fallback
- **EOD (End of Day)** per row: pre-block → flatten → straggler → confirm flat
- **SOD (Start of Day)** - optional entry gate before a configured hour (requires EOD enabled on row, because if there is no EOD, there is no need for SOD)
- **Daily drawdown** - monitoring with session-persistent baseline and optional proxy denominator
- **USD exposure cap** — global expected-loss ceiling at broker stop levels
- **Session persistence** — `session_state.json` survives Java restarts; broker reconciliation on startup.

### Key statistics

| Metric               | Value             |
|----------------------|-------------------|
| Java source files    | ~86 main + 7 test |
| Lines of Java (src/) | ~21,000           |
| HTTP port            | 8087              |

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| HTTP endpoints        | 9                                                 |
| Background schedulers | Recovery poll 10s; EOD poll 1s; session flush ~3s |
| Primary entry class   | <code>ZeroDeltaApplication.java</code>            |
| Primary engine        | <code>OhlcTradingEngine.java</code>               |

### *Technology stack*

| Layer               | Technology                                                  |
|---------------------|-------------------------------------------------------------|
| Language            | Java 17                                                     |
| Build               | Maven 3.x                                                   |
| UI                  | JavaFX 21                                                   |
| JSON                | Gson                                                        |
| Trading platform    | MetaTrader 5                                                |
| Transport           | HTTP (embedded <code>HttpServer</code> )                    |
| Default persistence | Local files under market data folder                        |
| Optional DB         | PostgreSQL JDBC (see §16 — not used for live command queue) |

### *In the code*

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| Entry point:        | <code>com.zerodelta.ZeroDeltaApplication</code>         |
| HTTP server:        | <code>com.zerodelta.server.ApplicationHttpServer</code> |
| EA source of truth: | <code>path<sub>mt5</sub>/EA_SOURCE.txt</code>           |

## 2. SYSTEM ARCHITECTURE

Think of MT5 as the **hands** and Java as the **brain**.

The brain never touches the broker directly; it writes instructions into a queue. The EA on the chart repeatedly asks, "Do you have a command for me?" (GET /getcommand). When something happens at the broker (fill, reject, close), the EA reports back (POST /executionfeedback). Java updates its picture of the world and only then sends the next instruction.

Your **Robot Matrix** is the control panel.

**Logs** (console + daily file) record decisions.

**Files** on disk store candles, layout, session state, and trade history snapshots.

### ARCHITECTURE DIAGRAM

---

#### MetaTrader 5 Terminal

---

ZeroDelta\_Ticks.mq5 (EA on chart)

- POST OHLC bars → /tickrecorder
- POST ticks → /tickdata
- POST account → /accountinfo
- POST trade snapshot → /tradehistory
- GET next command → /getcommand
- POST fill result → /executionfeedback

HTTP JSON, port 8087



---

#### ZeroDeltaApplication.java

---

ApplicationHttpServer  
Handlers → TradingOrchestrator (control thread)

---

#### Core services

---

|                           |                               |
|---------------------------|-------------------------------|
| OhlcTradingEngine         | Pattern / arm / IE / defer RT |
| ZeroDeltaRecoveryRegistry | IE↔RT twins                   |
| EodRowStateMachine        | EOD phases per row            |
| DrawdownMonitor           | Daily DD latch                |
| SessionReconciler         | Boot reconcile                |
| TradeCommandQueue         | RRC command queue             |
| EntryStrategyService      | IE command builder            |
| RecoveryTradeService      | RT geometry + command         |

---

#### UI (JavaFX)

---

|                     |                             |
|---------------------|-----------------------------|
| RobotMatrixTable    | Robot rows + master trading |
| ControlPanelPane    | DD / emergency controls     |
| ConnectionIndicator | Data stall indicator        |

---

#### Persistence (file DB mode)

---

market\_data/  
logs/log\_YYYY-MM-DD.txt  
session\_state.json  
robot\_layout.json

```
trade_history.json
{SYMBOL}_{TF}_candles.txt
```

### *Component layers*

| Layer       | Responsibility                                       | Key types                                     |
|-------------|------------------------------------------------------|-----------------------------------------------|
| UI          | Configure robots,<br>start/stop server, view<br>logs | RobotMatrixTable,<br>ControlPanelPane         |
| Service     | Trading logic, risk,<br>session, queues              | OhlcTradingEngine,<br>EodRowStateMachine, ... |
| Bridge      | HTTP handlers, EA<br>contract                        | mt5/*Handler,<br>tickrecorder/*               |
| Persistence | Files (+ optional<br>Postgres for history<br>reads)  | SessionStateStore,<br>AsyncFileWriter         |

### *In the code*

| Package                | Role                                 |
|------------------------|--------------------------------------|
| com.zerodelta          | Application shell                    |
| com.zerodelta.services | Business logic (~45% of main LOC)    |
| com.zerodelta.mt5      | EA HTTP bridge                       |
| com.zerodelta.ui       | JavaFX matrix                        |
| com.zerodelta.models   | DTOs: RobotConfig, TradeCommand, ... |
| com.zerodelta.server   | HTTP server bootstrap                |



### 3. DESIGN PHILOSOPHY

Zero Delta is built so that **the broker and Java never disagree for long**.

If Java thought it sent “buy” but never heard back, it would not send a “sell” on top – that causes duplicate or conflicting trades. Every important action follows **send** → **wait for confirmation** → **update state**.

**The broker is the final truth for open orders**, but Java must **remember intent** across restarts (session file). On startup, Java enters **reconciling** until MT5 sends a fresh trade history snapshot.

Recovery twins exist because IE and RT are linked bets: if IE wins (take profit), the RT pending order must disappear. If that link breaks, you can have a “ghost” RT order on the broker – documented in **the Appendix**.

The core principles are as summarised:

#### SEPARATION OF CONCERNS

| Concern                           | Owner                                              |
|-----------------------------------|----------------------------------------------------|
| Pattern detection, sizing, gates  | Java ( <code>OhlcTradingEngine, ...</code> )       |
| OrderSend, OrderDelete, positions | MT5 EA                                             |
| Operator configuration            | JavaFX matrix → <code>RobotConfig</code>           |
| Short-term open/pending view      | <code>TradeHistoryCache</code> (from EA snapshots) |

#### ***Request–Response–Confirmation (RRC)***

##### **Without RRC (failure mode):**

1. Java sends SELL; no feedback yet.
2. Java still sees “flat” in cache → sends BUY.
3. MT5 executes both → conflict.

##### **With RRC:**

1. Java enqueues command → `SENT` via `/getcommand`.
2. Further commands for same logical action blocked until feedback.
3. EA posts `/executionfeedback` → Java confirms → reconciles → may send next command.

#### ***Session-centric state***

Zero Delta’s **live command queue is in-memory** (`TradeCommandQueue`). Persistence is **linkage and intent**: recovery pairs, deferred RT, EOD phase, DD baseline (`session_state.json`).

### ***Defensive gates (layered)***

New entries must pass **all** applicable gates:

Master trading enabled

Session phase `READY` (not `RECONCILING`)

Daily DD not latched

EOD not blocking row

SOD not blocking row (if configured)

USD exposure cap (if enforced)

Pattern dedup / already executed key

### ***Twin integrity (IE + RT)***

RT is **never** placed before IE **position ticket** is known.

IE closed at TP → cancel RT pending (registry + history reconcile).

IE closed otherwise → cancel RT after short delay.

In case of delete failure → retry with backoff → `DELETE_ALL_MAGIC` fallback → [ALERT] log line.

*In the code*

#### **Principle**

RRC

Control sequencing

Twins

Session

EOD

#### **Primary class**

`TradeCommandQueue,`  
`ExecutionFeedbackHandler`

`TradingOrchestrator`

`ZeroDeltaRecoveryRegistry`

`SessionReconciler, SessionStateStore`

`EodRowStateMachine`

## 4. THE ROBOT MATRIX (OPERATOR CONTROL PANEL)

Each row is one robot: a symbol (e.g. XAUUSD), timeframe (e.g. H1), and a magic number that tags its trades at the broker. Rows can be active or not. Master trading is a global switch – if off, nothing fires.

*Settings you touch most often:*

**RPT %** – how much of balance to risk on IE stop distance.

**Recovery** – whether RT twin is created; **Rec %**, **Rec Dist %**, **Rec SL Off %** shape RT.

**EOD** – time to flatten and stop entries for that row.

**SOD** – earliest hour (MT5 time) new entries allowed next day (only with EOD on).

**Daily DD limit** – row/account loss cap for the MT5 calendar day.

Layout saves to: `robot_layout.json`;

Defaults save to: `robot_defaults.json`.

### OPERATIONAL FIELDS (*RobotConfig*)

| Field                                                                             | Operator meaning                        |
|-----------------------------------------------------------------------------------|-----------------------------------------|
| <code>active</code>                                                               | Row participates when master on         |
| <code>magicNumber</code>                                                          | IE magic ( $RT = \times 10000 + 3333$ ) |
| <code>symbol, timeframe</code>                                                    | Evaluation scope                        |
| <code>minX1Height, x1Type</code>                                                  | Minimum signal candle size              |
| <code>minGapPercent, gapFromX1ClosePercent</code>                                 | Gap between X2 and X1                   |
| <code>stopLossPercent, tp1Percent</code>                                          | IE bracket as % of X1 height            |
| <code>rptPercent, rptFixedAmount</code>                                           | Position sizing                         |
| <code>setupAEnabled, setupBEnabled</code>                                         | Strict vs relaxed X2/X1 pairing         |
| <code>recoveryEnabled, recoveryPercent, recoveryDistance, recoverySlOffset</code> | RT twin                                 |
| <code>eodEnabled, eodHour, eodMinute</code>                                       | End of day                              |
| <code>sodEnabled, sodStartHour</code>                                             | Start of day gate                       |
| <code>dailyDdLimit</code>                                                         | Per-row DD contribution / display       |

## 5. MARKET DATA & TIME

Robots do not react to every tick for entry logic (but they still matter for spread at X1 close (doji/body filter).

Robots react when a **candle closes** on their timeframe. That closed bar may become X1 or X2.

All **EOD, SOD, and daily DD day boundaries** use **MT5 platform time**, not your PC clock. If Java and MT5 disagree on “what day it is,” DD and EOD can look wrong until platform clock is synced.

### *How it works – data flow*

1. EA posts batch of closed bars → `POST /tickrecorder`
2. `CandleHistoryManager` stores per (symbol, timeframe)
3. On new close for (symbol, tf) → `OhlcTradingEngine.evaluatePatternOnClose`
4. Ticks → `POST /tickdata` → `SpreadSnapshotCache` for live spread at evaluation

| Component        | File                                                                     |
|------------------|--------------------------------------------------------------------------|
| OHLC ingress     | <code>OhlcDataHandler</code>                                             |
| Tick ingress     | <code>TickDataHandler</code>                                             |
| Candle store     | <code>CandleHistoryManager</code> ,<br><code>RollingCandleStorage</code> |
| Platform clock   | <code>Mt5PlatformClock</code>                                            |
| File persistence | <code>AsyncFileWriter</code> →<br><code>{SYMBOL}_{TF}_candles.txt</code> |

## 6. TRADING STRATEGY – X1 MOMENTUM ENTRY

The system looks for two consecutive candles (X2 then X1) where direction and geometry suggest a continuation trade on the next bar's open (X-open).

**Setup A** is stricter about X2/X1 shapes; **Setup B** is relaxed.

X1 must be large enough (min height), have a real body (not a doji vs spread), and sit in a minimum gap from X2. If all pass, the setup is armed – waiting for X-open to actually send the IE.

*How it works – strategy flow*

1. New candle closes on robot's (symbol, TF)
2. Identify X1 candidate; find X2 partner → pattern type (BUY / SELL)
3. Gates: EOD, SOD, DD, session RECONCILING, master trading
4. Filters: min X1 height, spread/doji, min gap
5. Pattern dedup – same X1 close not traded twice
6. ARM setup → wait for next bar forming\_open
7. At X-open → build IE → burst → feedback → (defer RT)

*Worked example (conceptual – XAUUSD BUY)*

Assume X1 height = 8.50

price units, stopLossPercent = 10%,

tp1Percent = 250%,

entry at X-open = 4500.00:

IE SL distance =  $8.50 \times 0.10 = 0.85 \rightarrow \text{SL} \approx 4499.15$

IE TP distance =  $8.50 \times 2.50 = 21.25 \rightarrow \text{TP} \approx 4521.25$

Lot size from `LotSizeCalculator` using RPT % and broker contract spec  
(Exact prices depend on buy/sell direction and pattern type.)

| Step          | Class / method                                                    |
|---------------|-------------------------------------------------------------------|
| Pattern       | <code>PatternDetector</code> , <code>PatternDetectionCache</code> |
| Evaluation    | <code>OhlcTradingEngine.evaluatePatternOnClose</code>             |
| Entry build   | <code>EntryStrategyService.buildEntryCommands</code>              |
| Decision logs | <code>TradeEvaluationLogger</code> → lines prefixed<br>`EVAL \`   |

## 7. INITIAL ENTRY (IE) & X-OPEN BURST

Instead of one click at X-open, the EA fires a **burst of market orders for a few seconds**. That improves fill chance when the open is fast or liquidity is thin. Java sets a **deadline**; if no position ticket arrives in time, the burst is aborted.

**Recovery (RT) is not sent with IE.** Java holds RT in memory until it **knows** the IE position ticket from execution feedback or trade history backfill.

### IE lifecycle

1. `EntryStrategyService` builds IE `PLACE_ORDER`
2. `XOpenBurstCommandFactory` wraps as `PLACE_ORDER_BURST` with TIF parameters
3. `TradeCommandQueue.addNormal` → EA polls `/getcommand`
4. `ExecutionFeedbackHandler` → `TradingOrchestrator` →  
`OhlcTradingEngine.onEntryExecutionFeedback`
5. On success/partial → resolve **position ticket** → `tryFinalizeRtAfterIeConfirmed`
6. `completeEntryState` clears armed / pending entry maps

### X-open TIF constants ('XOpenTifConfig')

| Constant                           | Value (milliseconds) | Meaning                                         |
|------------------------------------|----------------------|-------------------------------------------------|
| <code>TIF_DURATION_MS</code>       | 5000                 | Burst window length                             |
| <code>BURST_INTERVAL_MS</code>     | 50                   | Spacing between <code>OrderSend</code> attempts |
| <code>MAX_ATTEMPTS</code>          | 100                  | Hard cap on attempts                            |
| <code>TIF_WATCHDOG_GRACE_MS</code> | 500                  | Extra grace before watchdog abort               |

### Entry block reasons (search logs)

Log prefix: `EVAL | row=N ... | STOP or FAIL`

Common causes: `eod, sod, daily dd, master trading disabled, session reconciling, min_x1, min_gap, x1_spread_doji, usd_exposure_cap`

| Concern         | Class                                                   |
|-----------------|---------------------------------------------------------|
| Burst factory   | <code>XOpenBurstCommandFactory</code>                   |
| Deferred RT map | <code>OhlcTradingEngine.deferredRtByIeCommandId</code>  |
| IE feedback     | <code>OhlcTradingEngine.onEntryExecutionFeedback</code> |

## 8. RECOVERY TRADE (RT) & IE <-> RT TWINS

When recovery is enabled, each IE gets a **twin** RT: a pending stop order on the opposite side, anchored at the **IE stop price**. If IE hits **take profit**, the RT must be **cancelled** — you do not want a stray stop left on the chart. If IE hits **stop**, RT may trigger and become a position.

Java tracks twins in the **recovery registry**: IE ticket, RT pending ticket, RT position ticket. Every **10 seconds** (and on trade history updates), it checks for orphans or manual deletes at the broker.

### RT geometry

- Recovery magic:  $RT\_magic = IE\_magic \times 10000 + 3333$  (e.g IE magic 200 → RT magic 2003333).
- RT entry price = IE stop loss (stop order triggers if price reaches that level).
- RT TP/SL are derived from X1 bar geometry (`RecoveryTradeService`):
- BUY IE → RT is SELL\_STOP;
- Span = X1 close - X1 low;
- TP below X1 low by `recoveryDistance%`;
- SL above X1 close by `recoverySlOffset%` of span.
- SELL IE → RT is BUY\_STOP;
- Span = X1 high - X1 close;

RT lot size scales from IE fill and `recoveryPercent`.

### Twin lifecycle

IE placed (burst) → IE position ticket confirmed

#### THEN:

- RT PLACE\_ORDER queued → RT pending ticket recorded
- [IE open, RT pending]
- IE closes at TP → DELETE\_PENDING RT
- IE closes otherwise → DELETE\_PENDING RT (after ~3s poll path)
- RT triggered → RT position ticket recorded
- Both flat → pair removed from registry

### Orphan prevention (2026 hardening)

#### Failure

Single delete rejected by broker (e.g. retcode 10031)

Repeated delete failure

#### Mitigation

Clear dedup; retry with exponential backoff

Emergency `DELETE_ALL_MAGIC` on RT magic

|                                        |                                                                                   |
|----------------------------------------|-----------------------------------------------------------------------------------|
| Operator visibility                    | <code>[ALERT] RECOVERY ALERT row=N ...</code> in console<br>+ WARNING in log file |
| EOD missed pending                     | Flatten queues <b>delete RT magic before close</b> ;<br>dedupe emergency queue    |
| False EOD “flat”                       | CONFIRMED only if cache flat <b>and</b> registry<br>has no outstanding RT         |
| IE closed but TP not detected in cache | Reconcile on history update cancels RT when<br>IE not open                        |

See **Appendix C** for a real log trace.

### *Partial fill policy*

If IE fills below `PartialFillRtPolicy.MIN_IE_FILL_RATIO_FOR_RT` of requested size, RT is **skipped** (`RT_SKIPPED` in logs) and pair marked `rtSkippedPartial`.

| Concern         | Class                                                                      |
|-----------------|----------------------------------------------------------------------------|
| RT bracket math | <code>RecoveryTradeService.computeRtBracket</code>                         |
| Twin registry   | <code>ZeroDeltaRecoveryRegistry</code>                                     |
| Poll            | <code>recoveryPollScheduler 10s →</code><br><code>pollTwinStatus</code>    |
| Delete feedback | <code>onExecutionFeedback →</code><br><code>onDeletePendingFeedback</code> |

**Log key phrases:** `ZD Recovery`, `RECOVERY ALERT`, `DELETE_PENDING`, `DELETE_ALL_MAGIC`



## 9. END OF DAY (EOD) & START OF DAY (SOD)

**EOD** is “close shop” for a row: stop arming new trades, cancel queued entries, **close open positions**, and **delete pending orders** (including RT stops).

Timing uses **MT5 platform clock** at the row's `eodHour:eodMinute`.

**SOD** is the opposite morning rule: even after midnight, **do not start entries** until `sodStartHour` on the new MT5 day. SOD only applies when **both SOD and EOD are enabled** on that row.

*EOD phases (`EodRowStateMachine`)*

| Phase             | When                  | Actions                                                     |
|-------------------|-----------------------|-------------------------------------------------------------|
| PRE_BLOCK         | T - 5s                | Cancel queued<br>PLACE_ORDER; clear armed;<br>block entries |
| FLATTENING        | T                     | Emergency flatten pass                                      |
| STRAGGLER         | T + 5s                | Second flatten if not flat                                  |
| CONFIRMING        | Until flat            | Retry flatten every 30s                                     |
| CONFIRMED         | Cache + registry flat | Clear recovery + armed for<br>row                           |
| Midnight rollover | MT5 date change       | Reset CONFIRMED rows to<br>IDLE; daily state rollover       |

*Flatten command order (per row)*

1. DELETE\_ALL\_MAGIC — **RT magic** (pending stops)
2. EMERGENCY\_CLOSE\_ALL — RT magic (open RT positions)
3. DELETE\_ALL\_MAGIC — **IE magic**
4. EMERGENCY\_CLOSE\_ALL — IE magic

Commands use `addEmergencyIfAbsent` to avoid flooding the queue with duplicates.

**Important:** EMERGENCY\_CLOSE\_ALL closes **positions only**. Pending RT orders require **DELETE**  
- hence delete-first ordering.

*SOD (`SodTimingService`)*

Blocks entries when: `eodEnabled` && `sodEnabled` on row, and Current MT5 time is before  
`sodStartHour` on current MT5 calendar day

**Role**

## Class

EodRowStateMachine

EodTimingService

SodTimingService

Poll

Phase machine

Cutoff / pre-block / straggler timing

Morning gate

1s → TradingOrchestrator.onEodTick

**Log phrase:** EOD: row N

## 10. RISK MANAGEMENT

Each IE risks a **percentage of account balance** (RPT % or RPT\$ = fixed amount) to the stop loss distance. Bigger stop distance = smaller lot, for similar dollar risk.

**Daily drawdown (DD)** — Tracks how far **Today's P&L** has moved against you vs **start-of-day equity**. If loss exceeds **daily DD limit**, a **latch** blocks new entries until the next MT5 day (or stale latch cleared if equity recovers).

**Proxy balance** — When proxy doesn't equal 0, and is *enforced*, Daily Drawdown **percentage** may use proxy as denominator while P&L still comes from **live MT5 equity** (see matrix proxy settings).

**USD exposure cap** — Optional global limit on total expected loss at stop levels across open IE exposure.

### Drawdown

Baseline captured at MT5 day start or session restore

`DrawdownMonitor.setSessionBaseline/accountinfo`

Equity updates

→ `syncDailyDrawdownFromEquity`

`isDrawdownBreached()` checked in `OhlcTradingEngine` entry paths

**Persisted:** `dailyDdMt5Date`, `dailyDdStartOfDayEquity`, `dailyDdLiveEquityAtBaseline`, `dailyDdUseProxyBaseline`, `drawdownLimitBreached` in `session_state.json`

#### Concern

DD math / latch

Sizing

Exposure cap

UI sync

#### Class

`DrawdownMonitor`

`LotSizeCalculator`

`ZeroDeltaUsdExposureCap`

`RobotMatrixTable.syncDailyDrawdownFromEquity`

## 11. SESSION PERSISTENCE & RESTART RECOVERY

If you **restart Java**, the app does not forget everything.

Java reloads **robot layout**, **session state**, and warm **trade history** from disk.

On first connect, it waits for MT5 to send a **fresh broker snapshot** before allowing **new entries** – so it does not double-trade against positions you already have.

*What survives restart:*

- Recovery pair tickets (if saved)
- Deferred RT commands waiting for IE ticket
- EOD phase per row
- Daily DD baseline and latch
- Pattern dedup keys

*What does **not** survive restart:*

- In-memory command queue (unsent commands lost – by design)
- Armed setups not yet written to session (may be lost)

*Boot sequence*

1. Load robot\_layout.json, robot\_defaults.json
2. Warm trade\_history.json → TradeHistoryCache
3. Load session\_state.json → SessionReconciler.applySessionFile
4. Phase = RECONCILING → blocks new entries
5. First POST /tradehistory full snapshot → onBrokerSnapshot → READY
6. SessionStateStore debounced flush (~3s) while running `session\_state.json` (version 1) – *main sections*

| Key             | Purpose                                |
|-----------------|----------------------------------------|
| deferredRt      | RT waiting for IE confirmation         |
| recoveryPairs   | Active IE<->RT twin linkage            |
| eodPhase        | Per-row EOD state                      |
| dailyDd         | Drawdown baseline fields               |
| patternDedup    | Prevent duplicate pattern fires        |
| executedEntries | Exec keys already consumed             |
| nakedWatch      | Tickets needing emergency close backup |

**Class**`SessionReconciler``SessionStateStore``SessionStateDocument`**Broker adopt****Role**`RECONCILING → READY``Read/write JSON`**Schema**`ZeroDeltaRecoveryRegistry.adoptOpenPairFromBroker`

## 12. COMMAND QUEUE & EXECUTION FEEDBACK

Commands sit in a **queue**. The EA picks them up one at a time when it polls.

**Emergency** commands (EOD flatten, delete-all fallback) **jump ahead** of normal entry orders.

After the EA runs a command, **it must report success or failure to Java**.

Java marks the command confirmed or failed before relying on it.

### *RRC flow*

1. Java creates TradeCommand → TradeCommandQueue
2. EA GET /getcommand → receives CSV or burst JSON
3. EA executes → POST /executionfeedback
4. TradingOrchestrator.processFeedback
5. Recovery registry / engine / naked watch react
6. Next command eligible

### *Queue priority*

#### **Priority**

EMERGENCY

NORMAL

#### **Typical actions**

EMERGENCY\_CLOSE\_ALL,

DELETE\_ALL\_MAGIC

PLACE\_ORDER,

PLACE\_ORDER\_BURST,

DELETE\_PENDING\_TICKET,

CLOSE\_POSITION

### *Trade command actions*

#### **Action**

PLACE\_ORDER

PLACE\_ORDER\_BURST

DELETE\_PENDING\_TICKET

DELETE\_ALL\_MAGIC

EMERGENCY\_CLOSE\_ALL

CLOSE\_POSITION

MODIFY\_ORDER / CANCEL\_ORDER

#### **Purpose**

Single order

X-open IE burst (JSON to EA)

Cancel one pending order by ticket

Cancel all pending for magic + symbol

Close all positions for magic + symbol

Close one position

Order maintenance

**Class**

TradeCommandQueue

CommandPollHandler

ExecutionFeedbackHandler

TradingOrchestrator

**Role**

addNormal, addEmergency,  
addEmergencyIfAbsent, poll

GET /getcommand priority chain

POST /executionfeedback

Single-thread dispatch

### 13. TRADE HISTORY CACHE

Java keeps a **rolling window** of what MT5 last reported: open positions, pending orders, and recently closed trades. It is **not** a full brokerage statement.

EOD “is flat?” and recovery “is RT still pending?” consult the cache.

If the cache is **stale or incomplete**, Java can think the row is flat when MT5 still has a pending RT – or miss IE take-profit detection if the closed IE row aged out.

#### *Entry states*

| Detection     | State string contains                 |
|---------------|---------------------------------------|
| Open position | POSITION_OPEN or OPEN                 |
| Pending order | PLACED or STARTED (not CANCEL/REJECT) |
| Closed        | POSITION_CLOSED                       |

**Closed position retention time:** ~5 minutes (CLOSED\_RETENTION\_MS = 300\_000) then closed rows drop from cache.

| Method                         | Role                        |
|--------------------------------|-----------------------------|
| mergeSnapshot                  | Replace/update from EA POST |
| hasOpenOrPendingForMagicSymbol | EOD flat check              |
| isTicketPendingOrder           | Recovery RT pending check   |
| TradeHistoryHandler            | POST /tradehistory          |



## 14. HTTP API REFERENCE

All URLs are on `http://127.0.0.1:8087` (configure EA `JavaBaseURL`).

The EA **calls Java**; Java does not call the broker API directly.

When `/getcommand` returns `NO_COMMAND`, the EA continues its loop — posting data and polling again.

### Endpoint table

| Path                             | Method | Called by | Purpose                                     |
|----------------------------------|--------|-----------|---------------------------------------------|
| <code>/tickrecorder</code>       | POST   | EA        | OHLC bar batches                            |
| <code>/tickdata</code>           | POST   | EA        | Bid/ask ticks                               |
| <code>/getcommand</code>         | GET    | EA        | Poll next command or idle directive         |
| <code>/executionfeedback</code>  | POST   | EA        | Command result JSON                         |
| <code>/accountinfo</code>        | POST   | EA        | Balance, equity, currency, conversion rates |
| <code>/tradehistory</code>       | POST   | EA        | Open/pending/position snapshot              |
| <code>/platformclock</code>      | POST   | EA        | Ack stub (clock sync)                       |
| <code>/mt5pendingsnapshot</code> | POST   | EA        | Ack stub                                    |
| <code>/zdieietpclose</code>      | POST   | EA        | Ack stub (IE TP notification)               |

`/getcommand` idle priority (when queue empty)

1. Trade command from queue (if any)
2. `PING_WAKE` — if market data stall  $\geq \sim 12s$
3. `ZD_MATRIX_SCOPE, sym, tf, ...` — resend every  $\sim 30s$  or on scope change
4. `SEND_ACCOUNT_INFO, <uuid>` — if account stale  $\geq \sim 5s$
5. `NO_COMMAND`

## *Dispatch formats*

| Action            | Format                      |
|-------------------|-----------------------------|
| PLACE_ORDER_BURST | TradeCommand.toEaFlatJson() |
| Others            | TradeCommand.toCsv()        |

Sometimes the main trade has already closed, but the software **does not notice quickly enough** because of a timing problem in how it reads recent trade history (TradeHistoryCache).

So it **does not cancel** the RT pending order. That order stays live on MT5 even though the main trade is finished.

That leftover pending order a **“ghost order”** — it looks like it belongs to an active setup, but the setup is actually over.

## *Debug curl examples*

```
curl -s http://127.0.0.1:8087/getcommand
```

Expect NO\_COMMAND or a command line when queue non-empty.

## *In the code*

```
ApplicationHttpServer.start() registers contexts; port constant HTTP_PORT = 8087 in  
ZeroDeltaApplication.
```

## 15. METATRADER 5 EXPERT ADVISOR

The EA collects OHLC/ticks, polls commands, executes them, and posts feedback and trade history.

Attach **ZeroDelta\_Ticks.mq5** to a chart. Enable **Allow WebRequest** for `http://127.0.0.1:8087`. Set **JavaBaseURL** in EA inputs. Start the Java app and press **Start HTTP Server** on the Tick Recorder tab.

### *EA responsibilities*

| Function            | HTTP                   |
|---------------------|------------------------|
| Publish bars/ticks  | POST                   |
| Poll commands       | GET                    |
| Report fills/errors | POST executionfeedback |
| Report account      | POST accountinfo       |
| Report open book    | POST tradehistory      |

### *Required command support (Java → EA)*

Java may send at minimum:

PLACE\_ORDER\_BURST (JSON)  
PLACE\_ORDER, CLOSE\_POSITION, CANCEL\_ORDER, MODIFY\_ORDER (CSV)  
EMERGENCY\_CLOSE\_ALL, DELETE\_ALL\_MAGIC, DELETE\_PENDING\_TICKET

## 16. DATA STORAGE & CONFIGURATION

The Default storage mode is local folder. This stores everything under your chosen **market data folder**: candles, logs, robot layout, session, and trade history JSON.

### *File mode layout*

|                         |                                              |
|-------------------------|----------------------------------------------|
| {marketDataFolder}/     |                                              |
| logs/log_YYYY-MM-DD.txt | daily file log (WARNING includes ALERT text) |
| session_state.json      | control plane snapshot                       |
| robot_layout.json       | matrix rows                                  |
| robot_defaults.json     | default field values                         |
| trade_history.json      | last persisted history                       |
| XAUUSD_M30_candles.txt  | per symbol/TF candle files                   |

### *UI startup*

1. DatabaseConfigDialog — file vs postgres path selection
2. Logger.configure(marketDataRoot)
3. Tabs: Login → Tick Recorder (server) → Matrix

### *PostgreSQL*

A **PostgreSQL** connection option exists for some history/read paths. **Live trade commands are not stored in the database** — the operational queue is in-memory. Details omitted here; file mode is the supported operational path.

| Class                  | Role                               |
|------------------------|------------------------------------|
| DatabaseConfigDialog   | Mode selection                     |
| TradeHistoryFileWriter | File history                       |
| PostgresClient         | JDBC (limited — no command insert) |

## 17. CONCURRENCY & THREAD SAFETY

Market data arrives on **many HTTP threads**, but **trading decisions** run on a **single control thread** so two parts of the program never arm and fire the same row at once. Row **locks** protect feedback and EOD from interleaving on the same robot row.

### Thread / executor

HTTP pool (10)

`TradingOrchestrator` control executor

`recoveryPollScheduler`

`eodPollScheduler`

JavaFX UI thread

`AsyncFileWriter`

### Role

Handlers accept POST/GET

Serializes feedback, history, EOD, recovery side effects

10s twin poll

1s EOD tick

Matrix updates via `Platform.runLater`

Disk candle writes

### Class

`TradingOrchestrator`

`RowLockRegistry`

`ConcurrentHashMap`

### Role

`controlExecutor`

Per-row 200ms try-lock

Shared caches (history, candles, queue dequeues)

## 18. LOGGING, MONITORING & ALERTS

In the log file (**logs/log\_YYYY-MM-DD.txt**), Important prefixes tell you what subsystem spoke.

When recovery escalates to delete-all fallback, look for **[ALERT]** – that is deliberate high visibility.

### Log prefixes

| Prefix                                   | Meaning                                           |
|------------------------------------------|---------------------------------------------------|
| <code>`EVAL \</code>                     | Pattern / filter decision on bar close            |
| <code>`EXEC \</code>                     | Entry execution / RT queue / feedback             |
| <code>ZD Recovery</code>                 | Twin registry actions                             |
| <code>[ALERT] RECOVERY<br/>ALERT</code>  | RT delete failed repeatedly;<br>DELETE_ALL queued |
| <code>EOD: row N</code>                  | End-of-day state machine                          |
| <code>GET /getcommand:</code>            | Command dispatched to EA                          |
| <code>EXEC FEEDBACK:</code>              | EA result JSON (truncated in console)             |
| <code>Trade history<br/>received:</code> | Cache open/pending counts                         |

### Operator watch list

| Watch for                                                  | Action                                             |
|------------------------------------------------------------|----------------------------------------------------|
| <code>RECOVERY ALERT</code>                                | Open MT5 – verify RT pending gone after DELETE_ALL |
| <code>EOD: row N retry flatten + high<br/>pending=N</code> | Pending orders not clearing – check RT magic       |
| <code>RECONCILING never clears</code>                      | EA not posting /tradehistory                       |
| <code>daily dd / latch messages</code>                     | Entries blocked until new day or equity recovery   |
| <code>EVAL ... eod FAIL</code>                             | Normal near EOD – not a bug                        |

| Component                          | Role                     |
|------------------------------------|--------------------------|
| <code>CircularLogBuffer</code>     | UI console throughput    |
| <code>Logger.warning</code>        | File log for ALERT lines |
| <code>TradeEvaluationLogger</code> | Structured EVAL/EXEC     |

## 19. TROUBLESHOOTING

### *Symptom-first guide*

#### **RT pending still on chart after IE take profit**

|                   |                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cause</b>      | Delete rejected once and old code never retried; or EOD only closed positions not pending; or registry dropped pair while RT in flight |
| <b>Check logs</b> | <code>ZD Recovery: IE TP, DELETE_PENDING, RECOVERY ALERT, retcode=10031</code>                                                         |
| <b>Check MT5</b>  | Pending order magic = RT magic (e.g. 2003333)                                                                                          |
| <b>Code</b>       | <code>ZeroDeltaRecoveryRegistry,<br/>EodRowStateMachine.queueFlatten</code>                                                            |

#### **EOD retry flatten forever**

|                   |                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------|
| <b>Cause</b>      | Cache still shows pending/open for IE or RT magic                                              |
| <b>Check logs</b> | <code>cache open=N pending=M, only<br/>EMERGENCY_CLOSE_ALL magic=200 without<br/>delete</code> |
| <b>Fix path</b>   | Ensure EA executes <code>DELETE_ALL_MAGIC</code> ; verify post-hardening flatten order         |

#### **No entries at all**

|                   |                                                                              |
|-------------------|------------------------------------------------------------------------------|
| <b>Cause</b>      | Master off; DD latched; RECONCILING; EOD/SOD gate; no pattern                |
| <b>Check logs</b> | <code>EVAL ... STOP reason; session phase</code>                             |
| <b>Code</b>       | <code>OhlcTradingEngine,<br/>SessionReconciler.blocksSessionEntries()</code> |

### EA not receiving commands

|               |                                                        |
|---------------|--------------------------------------------------------|
| <b>Cause</b>  | HTTP server stopped; wrong port;<br>WebRequest blocked |
| <b>Test</b>   | <code>curl http://127.0.0.1:8087/getcommand</code>     |
| <b>Config</b> | EA JavaBaseUrl=http://127.0.0.1:8087                   |

### Duplicate or conflicting trades

|              |                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------|
| <b>Cause</b> | RRC bypass — commands sent before<br>feedback (should not happen in normal paths)               |
| <b>Check</b> | Command queue depth; pending confirmation<br>state                                              |
| <b>Code</b>  | TradeCommandQueue,<br>PendingCommandsManager (conceptual — Zero<br>Delta uses queue + feedback) |

### Common matrix issues

| Issue                    | Cause                                     | Solution                                                            |
|--------------------------|-------------------------------------------|---------------------------------------------------------------------|
| Robot skipped — no magic | Empty magic column                        | Set unique magic per row                                            |
| Recovery not firing      | Recovery disabled or partial<br>fill skip | Enable recovery; check<br>RT_SKIPPED logs                           |
| DD dialog on startup     | Stale latch from session                  | Verify live equity vs limit; stale<br>latch cleared if within limit |



## 20. APPENDIX — CODE MAP (MAINTAINER INDEX)

| Symptom / topic    | Start here                                                                             |
|--------------------|----------------------------------------------------------------------------------------|
| Pattern not arming | <code>OhlcTradingEngine.evaluatePatternOnClose,</code><br><code>PatternDetector</code> |
| IE burst / TIF     | <code>XOpenBurstCommandFactory,</code> <code>XOpenTifConfig</code>                     |
| RT geometry wrong  | <code>RecoveryTradeService</code>                                                      |
| Twin orphan        | <code>ZeroDeltaRecoveryRegistry</code>                                                 |
| EOD stuck          | <code>EodRowStateMachine,</code> logs EOD:                                             |
| DD latch           | <code>DrawdownMonitor,</code> <code>RobotMatrixTable</code>                            |
| Session restart    | <code>SessionReconciler,</code> <code>session_state.json</code>                        |
| HTTP contract      | <code>ApplicationHttpServer,</code> <code>mt5/*Handler</code>                          |
| Matrix UI          | <code>RobotMatrixTable</code>                                                          |
| Command format     | <code>TradeCommand.toCsv,</code> <code>toEaFlatJson</code>                             |

### *Largest source files*

| File                                        | ~LOC | Role               |
|---------------------------------------------|------|--------------------|
| <code>RobotMatrixTable.java</code>          | 3300 | Matrix UI + config |
| <code>ZeroDeltaApplication.java</code>      | 2000 | Wiring + lifecycle |
| <code>OhlcTradingEngine.java</code>         | 1200 | Entry engine       |
| <code>ZeroDeltaRecoveryRegistry.java</code> | 900  | Twins              |